A Classical Image-Processing Pipeline for Crack Detection and Measurement on Concrete Surfaces

[Author name]
[Department, Institution]
kavishnasta@gmail.com
October 2026

***Abstract*—**Cracks are an early indicator of structural deterioration in concrete, but manual inspection is slow, subjective and hard to repeat. Most recent automated approaches rely on deep learning, which needs large labelled datasets and offers little insight into *how large* a crack is. This paper presents a fully classical, training-free image-processing pipeline that detects cracks in concrete photographs and measures the length and width of every individual crack. The pipeline combines contrast-limited adaptive histogram equalization, a median filter, the morphological black-hat transform, a thresholding stage with an absolute contrast floor, shape-based removal of specks and stains, skeletonization, and a distance-transform width estimate, with an optional conversion from pixels to millimetres using a reference object. Only two scalar parameters, the black-hat kernel size and the contrast floor, are selected by grid search on a validation set that is disjoint from the test images. On 2,000 held-out images from the METU / Özgenel concrete-crack dataset (1001 cracked, 999 clean), the method reaches an image-level accuracy of 95.5% (95% CI 94.4–96.3%), with precision 94.5%, recall 96.5% and specificity 94.4%. Under an identical cleanup and decision rule, it outperforms global Otsu thresholding (41.9% accuracy) and a validation-tuned Canny edge detector (92.8%). Error analysis shows that the remaining failures are dominated by very wide cracks that exceed the structuring element and by heavily textured or pitted surfaces; several apparent false alarms are visible cracks in images labelled clean. Pixel-level evaluation and validation of the measured widths against physical ground truth are left for future work.

***Keywords*—**crack detection; concrete; mathematical morphology; black-hat transform; skeletonization; distance transform; non-learning methods

1. INTRODUCTION

Cracking is one of the most common and most informative forms of damage in concrete structures such as walls, pavements and bridges. The width, length and growth of cracks indicate loading, shrinkage or corrosion problems, and undetected cracks allow water and chlorides to reach reinforcement. Routine inspection is still largely visual: an inspector walks the structure, photographs defects and judges their severity by eye. This is slow, depends on the inspector, and produces measurements that are hard to compare between visits [1].

Automatic detection from photographs has therefore attracted considerable attention. Recent work is dominated by convolutional neural networks, which classify image patches or segment cracks pixel by pixel [2], [3]. These models are accurate but need many labelled images, behave as black boxes, and typically report *where* a crack is rather than *how long and how wide* it is, which is the quantity engineers act on. Classical image processing needs no training data, runs in milliseconds on a CPU, and every step can be inspected, but it is generally regarded as fragile on real, textured concrete [1].

This paper asks how far a carefully engineered classical pipeline can go. We describe and evaluate a pipeline that (i) detects thin dark structures with the black-hat transform, (ii) suppresses false alarms from concrete texture with an absolute contrast floor and shape-based component filtering, and (iii) measures the length and width of every crack separately from its skeleton and distance transform, optionally in millimetres. Our contributions are:

1. an integrated, fully reproducible, non-learning pipeline that outputs per-crack length and width, not only a mask or a label;

2. an analysis showing why a thresholding stage without an absolute contrast floor fails on clean concrete, and a simple remedy;
3. a leakage-free evaluation protocol on the METU concrete-crack dataset [12], in which the only two tuned parameters are chosen on validation images disjoint from the test images, with per-image predictions released; and
4. an error analysis that identifies the failure modes of the approach, including apparent label noise in the dataset.

None of the individual operators is new; the contribution is their combination, the measurement stage, and the careful evaluation. The remainder of the paper is organised as follows. Section 2 reviews related work, Section 3 describes the method, Section 4 the experimental protocol, Section 5 the results, Section 6 discusses limitations, and Section 7 concludes.

2. RELATED WORK

Classical methods. Early crack detectors apply global or local thresholding [5], edge detection [6], and morphological operations [8], [9] to grayscale photographs, often followed by connected-component analysis. A critical review of such image-processing approaches [1] identifies illumination variation, surface texture and shadows as the main causes of failure, which motivates the contrast normalization and shape filtering used here.

Learning-based methods. Random structured forests were used for pixel-level road-crack detection and introduced the CrackForest dataset [4]. Deep convolutional networks have since been applied to road and building cracks [2], and pretrained CNNs reach very high classification accuracy on the METU patch dataset used in this paper [3]. We do not compete with these methods on raw accuracy; we study how much can be achieved without training and with direct, interpretable measurements.

Crack measurement. Estimating crack width from a binary mask is commonly done with a skeleton and the distance transform, and length by summing skeleton steps. We follow this standard recipe and make the pixel-counting conventions explicit, since single-pixel biases matter for cracks that are only a few pixels wide.

3. METHOD

Figure 1 shows the pipeline on one image. Given a colour photograph, the system outputs a binary crack mask, a skeleton, and, for every connected crack, its length, mean width and maximum width.

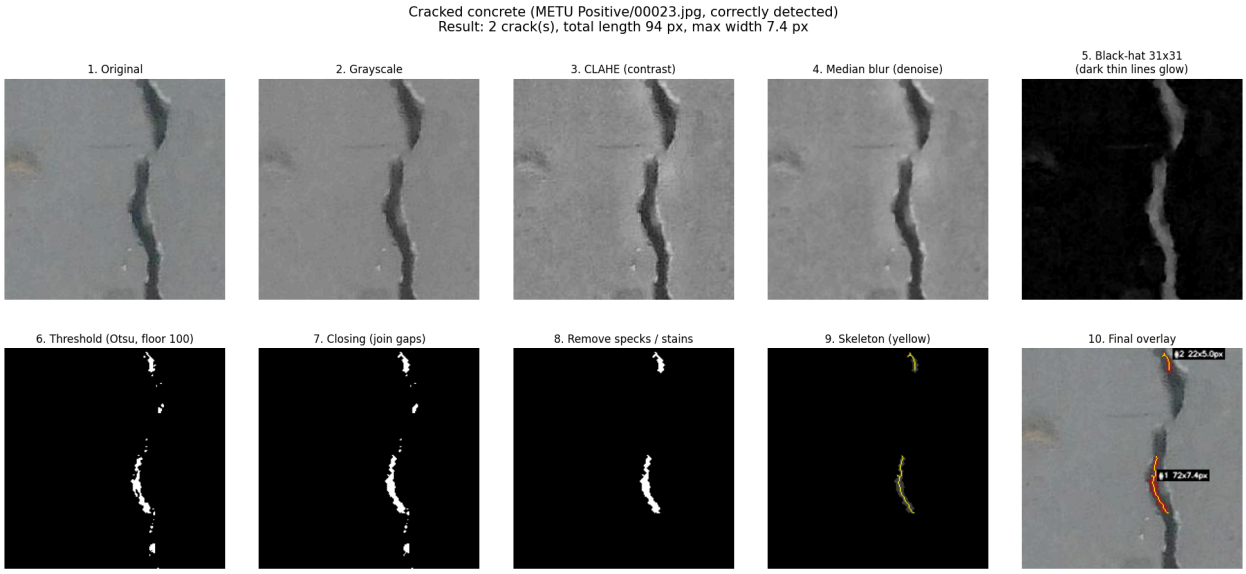


Fig. 1. Pipeline stages on a cracked METU image (Positive/00023.jpg): (1)–(4) preprocessing; (5) black-hat response; (6) threshold; (7) closing; (8) removal of specks and stains; (9) skeleton; (10) final overlay with crack id, length and maximum width in pixels. The crack is detected but split in two segments by the thresholding.

3.1 Preprocessing

The image is converted to grayscale. Images whose longer side exceeds 2000 px are first shrunk with area interpolation so that processing time stays bounded; the scale factor is recorded so that physical measurements remain correct (Section 3.6). We then apply contrast-limited adaptive histogram equalization (CLAHE) [7] with clip limit 1.0 and an 8×8 tile grid to even out illumination, followed by a 3×3 median filter to suppress sensor grain while preserving thin dark lines.

3.2 Black-hat highlighting

Cracks are thin, dark structures on a brighter background. The morphological black-hat transform [8], [9] returns the difference between the morphological closing of the image with a structuring element B and the image itself:

$$\mathrm{BH}(x) = (I \bullet B)(x) - I(x) \tag{1}$$

Dark features narrower than B are filled by the closing and therefore appear bright in BH, while broad background variation is removed. We use an elliptical B whose diameter (the "kernel size") is a tuned parameter: it must exceed the width of the widest crack to be detected, but a larger element also responds to broader dark texture.

3.3 Thresholding with an absolute contrast floor

The black-hat image is binarised with Otsu's method [5], but never below a fixed contrast floor τ:

$$T = \max\left(T_{\mathrm{Otsu}}, \tau\right), \qquad M(x) = 1 \text{ if } \mathrm{BH}(x) \geq T \tag{2}$$

The floor is essential. Otsu's method always splits the histogram into two classes, even when the image contains no crack at all, so on clean but grainy concrete it selects a low threshold and labels texture as foreground. CLAHE amplifies this effect. In a preliminary run on 100 clean and 100 cracked METU images (not part of the evaluation below), a pipeline with Otsu and a floor of 12 flagged 99 of the 100 clean images as cracked. Setting τ as an absolute threshold on the black-hat response, selected on validation data, removes this failure.

3.4 Cleanup

A morphological closing with an elliptical 3×3 element, applied twice, joins fragments of one crack. Connected components (8-connectivity) are then removed unless *all* of the following hold, where A is the component area in pixels and s the number of pixels of its skeleton: $A \geq 50$; $s \geq 15$; elongation $\eta \geq 1.5$; and mean width $\bar{w} \leq 20$ px, with

$$\eta = \frac{s}{\sqrt{A}}, \qquad \bar{w} = \frac{A}{s} \tag{3}$$

Elongation is high for long thin cracks and low for compact blobs; mean width is large for thick stains. Together they reject specks, pits and round stains without needing a separate classifier.

3.5 Measurement

The cleaned mask is skeletonized with the thinning algorithm of Zhang and Suen [10] as implemented in scikit-image [16]. Each 8-connected component of the mask is one crack and receives an identifier. **Length.** Along the skeleton of one crack, a horizontal or vertical step between neighbouring skeleton pixels counts one pixel and a diagonal step counts $\sqrt{2}$, so with N_o orthogonal and N_d diagonal steps

$$L = N_o + \sqrt{2} N_d \tag{4}$$

A diagonal link is ignored when its two pixels are already connected through an orthogonal neighbour, so the corner of a staircase is not counted twice. **Width.** Let D be the Euclidean distance transform [11] of the crack mask, i.e. the distance of each foreground pixel to the nearest background pixel. At a skeleton pixel p the local width is

$$w\left(p\right) = 2D\left(p\right) - 1, \qquad w_{\text{mean}} = \frac{1}{|S|} \sum_{p \in S} w\left(p\right), \qquad w_{\max} = \max_{p \in S} w\left(p\right) \tag{5}$$

where S is the crack's skeleton. The usual estimate is $2D$; we subtract one pixel because D is measured between pixel centres, so a bar that is w pixels wide (odd w) has $D = (w+1)/2$ at its centre and $2D$ would overestimate every width by one pixel, which is a 20–30% error for 3–5 px cracks. Unit tests on drawn bars of known width and length confirm the convention (widths within 1 px, length within a few pixels; skeletonization shortens a line by roughly half its width at each end, so lengths are slightly underestimated).

3.6 Conversion to physical units

If the user provides a scale σ in millimetres per pixel, lengths and widths are multiplied by it. Alternatively, the user clicks two points q_1, q_2 on a reference object of known length ℓ_{ref} in the photograph:

$$\sigma = \frac{\ell_{\text{ref}}}{\|q_1 - q_2\|}, \qquad L_{\text{mm}} = \sigma L, \qquad w_{\text{mm}} = \sigma w \tag{6}$$

If the image was downscaled by a factor $\rho < 1$ before processing, the scale is divided by ρ. Calibration with a single reference length assumes a planar surface viewed approximately perpendicularly, which we discuss as a limitation.

3.7 Image-level decision

For classification experiments, an image is declared cracked if it contains at least one crack of length at least $L_{\min} = 15$ px (L_c is the length of crack c):

$$\hat{y} = 1 \Leftrightarrow \max_{c} L_c \geq L_{\min} \tag{7}$$

If no crack passes the filters the system reports explicitly that no cracks were found, and unreadable files are skipped with a message.

4. EXPERIMENTAL SETUP

4.1 Dataset

We use the METU / Özgenel concrete-crack dataset [12], [3]: 40,000 colour images of 227×227 pixels, 20,000 labelled *Positive* (cracked) and 20,000 labelled *Negative* (clean), cropped from high-resolution photographs of building surfaces. The labels are image-level only, so this dataset supports evaluating *detection* but not the pixel-level quality of masks or the accuracy of widths.

4.2 Protocol

All 40,000 images are shuffled with a fixed random seed (42). The first 400 form a **validation set** (186 cracked, 214 clean) used only for parameter selection; the next 2,000 form the **evaluation set** (1001 cracked, 999 clean) and are never used for tuning. A grid search covers kernel sizes {15, 21, 25, 31} and contrast floors $\tau \in \{50, 60, \ldots, 120\}$ (32 settings), selecting the setting with the highest validation F1. All other parameters are fixed a priori (Table 1). Selecting two scalars on a validation split is hyperparameter selection, not learning from labelled examples of cracks.

TABLE 1
Pipeline parameters.

Parameter	Value	How set
CLAHE clip limit / tile grid	1.0 / 8×8	preliminary, fixed
Median filter size	3×3	fixed
Black-hat kernel (elliptical)	31×31	validation grid
Contrast floor τ	100	validation grid
Closing element / iterations	3×3 / 2	fixed
Min. area A; min. skeleton s	50 px; 15 px	fixed
Min. elongation η; max. mean width	1.5; 20 px	fixed
Min. crack length $L_{\min}$	15 px	fixed

4.3 Metrics and statistics

"Cracked" is the positive class. We report accuracy, precision, recall (sensitivity), specificity and F1, with the confusion matrix. Uncertainty in proportions is expressed with 95% Wilson score intervals [13]. When two methods are compared on the same images we use the exact McNemar test [14] on the discordant pairs.

4.4 Baselines and ablations

Two baselines replace only the detection stage and share the cleanup, filters and decision rule with the proposed method: (i) global Otsu thresholding [5] of the blurred grayscale image, taking the dark class as crack, and (ii) Canny edge detection [6] whose two hysteresis thresholds are tuned on the validation set. The ablation study removes one component at a time from the tuned pipeline: CLAHE, the median filter, closing, the component filter, and the Otsu step (a fixed floor only), and additionally evaluates the default 15×15 kernel.

4.5 Implementation

The pipeline uses Python 3.12 with OpenCV 4.11 [15], scikit-image 0.26 [16], NumPy [17], pandas and Matplotlib; no machine-learning library is used. The code, configuration, per-image predictions and figures are available at github.com/kavishnasta/concrete-crack-detector.

5. RESULTS

5.1 Parameter selection

Table 2 and Fig. 2 show validation F1 over the grid. The best setting is a 31×31 kernel with $\tau = 100$ (F1 = 0.957, accuracy 96.0% on 400 images). The surface is smooth: for each kernel, F1 rises with τ up to a maximum and then falls as real cracks are suppressed, and the optimal τ grows with kernel size because larger structuring elements produce stronger responses on texture. The default 15×15 kernel never exceeds an F1 of 0.891, because it cannot bridge the dark centre of the wide cracks that are common in this dataset. The optimum lies on the edge of the kernel grid, so larger kernels were not explored.

TABLE 2
Validation F1 over the tuning grid (kernel size × contrast floor τ; best in bold; 400 validation images).

Kernel \ τ	50	60	70	80	90	100	110	120
15×15	0.885	0.891	0.842	0.810	0.776	0.701	0.635	0.564
21×21	0.860	0.899	0.905	0.919	0.909	0.879	0.827	0.786
25×25	0.836	0.886	0.908	0.938	0.944	0.937	0.924	0.891
31×31	0.807	0.857	0.901	0.918	0.943	**0.957**	0.948	0.942

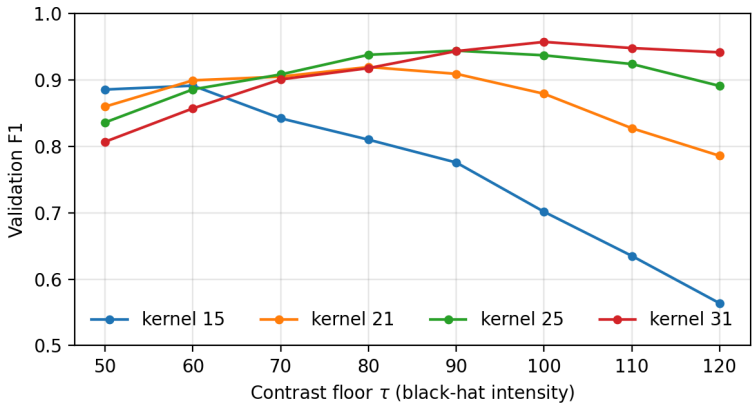


Fig. 2. Validation F1 versus contrast floor τ for each black-hat kernel size.

5.2 Held-out performance

On the 2,000 held-out images, the tuned pipeline classifies 1,909 correctly (Table 3, Fig. 3): accuracy 95.5% (95% CI 94.4–96.3%), precision 94.5%, recall 96.5%, specificity 94.4% and F1 95.5%. Of the 91 errors, 56 are false alarms and 35 are missed cracks. Held-out accuracy (95.5%) is close to validation accuracy (96.0%), which indicates that selecting two parameters did not overfit the validation images.

TABLE 3
Held-out evaluation on 2,000 METU images (kernel 31×31, $\tau = 100$). Intervals are 95% Wilson CIs.

	Predicted cracked	Predicted clean	Total
Actually cracked	966 (TP)	35 (FN)	1001
Actually clean	56 (FP)	943 (TN)	999

Metric	Count	% [95% CI]
Accuracy	1909 / 2000	95.5 [94.4, 96.3]
Precision	966 / 1022	94.5 [93.0, 95.8]
Recall (sensitivity)	966 / 1001	96.5 [95.2, 97.5]
Specificity	943 / 999	94.4 [92.8, 95.7]
F1 score	–	95.5

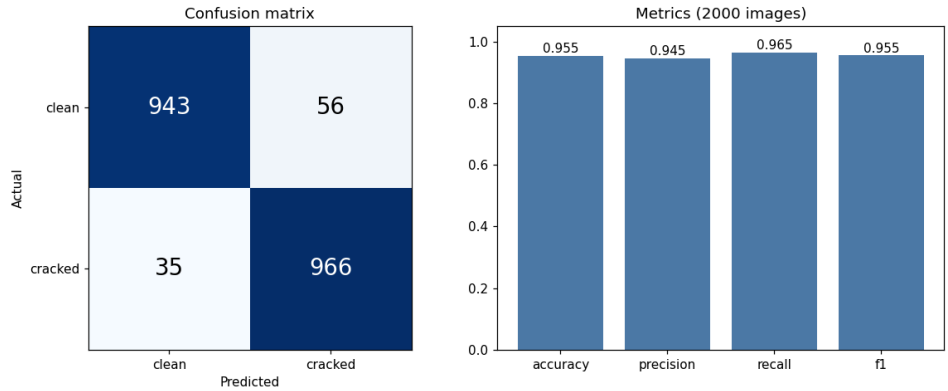


Fig. 3. Confusion matrix and summary metrics on the held-out evaluation set.

5.3 Comparison with baselines

Table 4 compares the proposed method (95.5% accuracy) with Global Otsu (41.9%, a difference of +53.6 points; McNemar p <0.001) and Canny (130/390) (92.8%, a difference of +2.6 points; McNemar p <0.001). All methods use the identical closing, component filter and decision rule, and the Canny thresholds were tuned on the same validation images, so differences are attributable to the detection stage. The global Otsu baseline has no free parameter and no illumination handling, so it should be read as a lower bound rather than a competitive method; the tuned Canny detector is the more meaningful comparison. Mean processing time per 227×227 image (Python implementation on the authors' machine, including the shared cleanup and measurement) was 21 ms for proposed, 13 ms for global otsu, 8 ms for canny.

TABLE 4

Comparison with baselines on the same held-out images, using the same cleanup and decision rule (accuracy with 95% CI; McNemar exact p versus the proposed method).

Method	Accuracy % [95% CI]	Precision %	Recall %	F1 %	McNemar p
Proposed (black-hat)	95.5 [94.4, 96.3]	94.5	96.5	95.5	–
Global Otsu	41.9 [39.8, 44.1]	45.6	82.2	58.6	<0.001
Canny (130/390)	92.8 [91.6, 93.9]	91.2	94.8	93.0	<0.001

5.4 Ablation study

Draft note — pending experiment. The ablation study has not been run yet. Run `python experiments.py` and rebuild this paper; this section and table then fill in automatically from `results/ablation.csv`.

TABLE 5

Ablation: one component removed at a time from the tuned pipeline (held-out images).

Variant	Accuracy %	Precision %	Recall %	F1 %
Full method	*pending*	*pending*	*pending*	*pending*
− CLAHE	*pending*	*pending*	*pending*	*pending*
− median blur	*pending*	*pending*	*pending*	*pending*
− morphological closing	*pending*	*pending*	*pending*	*pending*
− component filter	*pending*	*pending*	*pending*	*pending*
− Otsu (fixed floor only)	*pending*	*pending*	*pending*	*pending*
Kernel 15 instead of tuned	*pending*	*pending*	*pending*	*pending*

5.5 Qualitative results and error analysis

Figure 1 shows a correctly detected crack. Figure 4 shows the first four false alarms and the first four misses in file order, without hand-picking. The false alarms occur on speckled aggregate with dark pits (Negative/00143, 01332), on strongly ridged surfaces (01234), where many short dark lines form elongated components, and on an image (00283) that visibly contains a thin crack although it is labelled clean; the dataset therefore contains label noise, and the true specificity is probably somewhat higher than the measured value. The four missed cracks are all very wide, dark openings. Two design choices plausibly suppress them: such regions can exceed the structuring element, so the black-hat response inside them collapses, and the maximum mean width of the component filter (20 px) is designed to reject thick stains. We did not isolate which factor dominates; this is a hypothesis to be tested with the ablation and a multi-scale kernel.

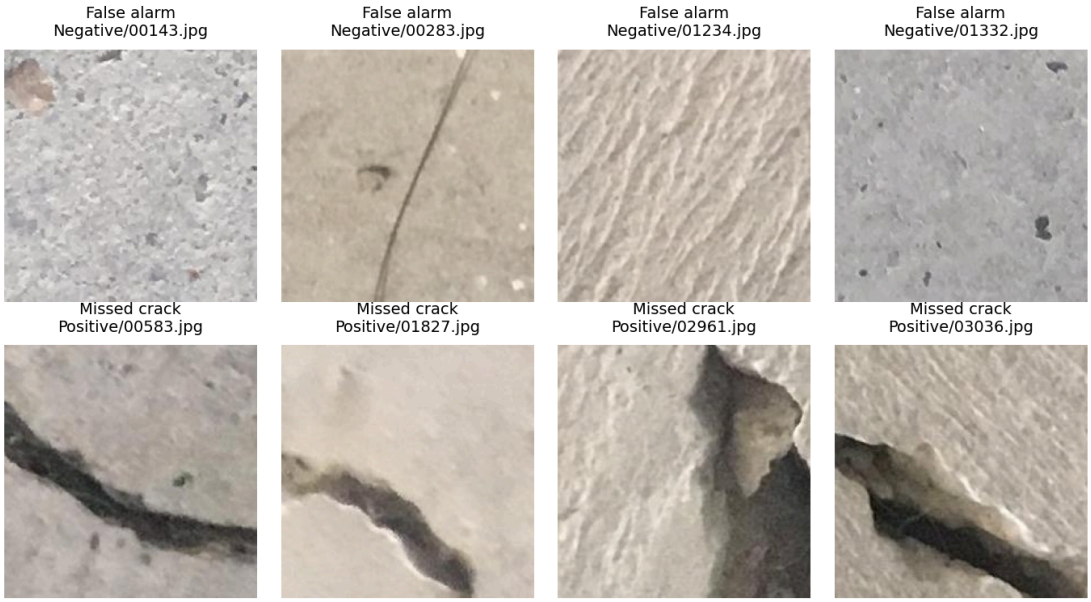


Fig. 4. First four false alarms (top) and first four missed cracks (bottom) in file order. Image names are METU file names.

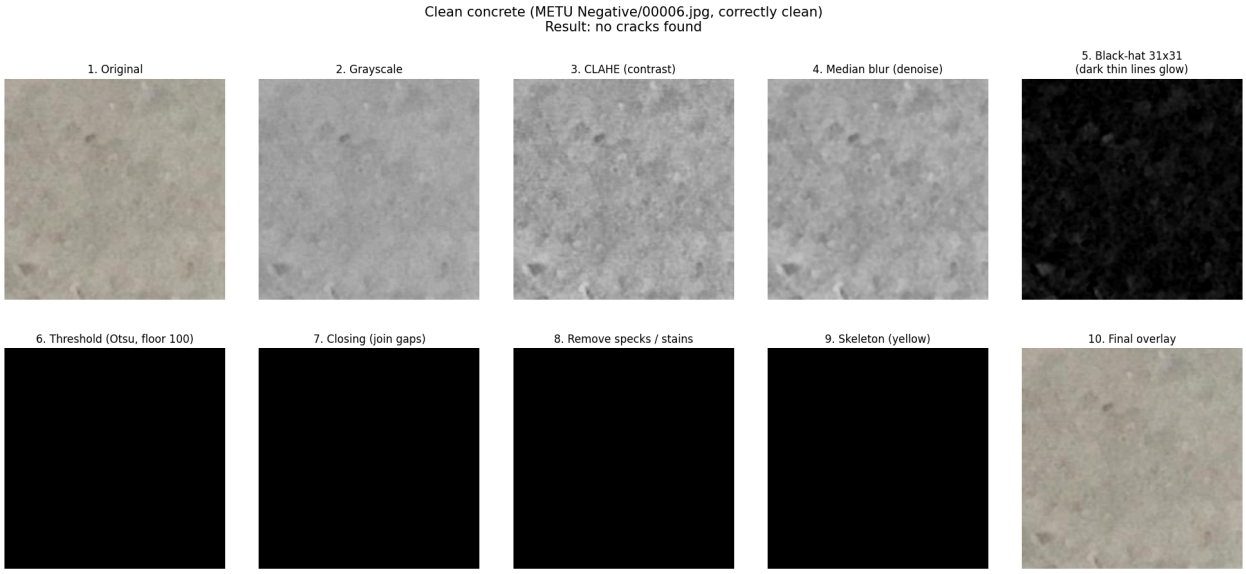


Fig. 5. Pipeline stages on a clean METU image (Negative/00006.jpg) that is correctly classified: small dark blobs survive thresholding but are removed by the shape filters, and no crack is reported.

6. DISCUSSION AND LIMITATIONS

What the results show. A classical pipeline with two tuned parameters classifies cracked and clean concrete patches with 95.5% accuracy and provides per-crack geometry. The main technical lesson is that thresholding on a *relative* criterion (Otsu alone) is unsafe for detection, because it cannot say "nothing here"; an absolute contrast floor is what makes the no-crack case work.

Limitations.

- *Image-level labels only.* METU supports detection accuracy but not mask quality. Pixel-level metrics (Dice, IoU) on CrackForest [4] with a small spatial tolerance are planned but not reported here.
- *Measurements are not validated against physical ground truth.* Length and width conventions were checked only on synthetic bars of known size in unit tests; no photographs with measured crack widths were used, and widths of a few pixels are limited by image resolution and by the 1 px convention above.
- *Possible split leakage.* The dataset consists of patches cropped from a few hundred larger photographs, and our split is random over patches, not over source photographs. Patches from the same photograph can appear in both validation and evaluation sets. With only two tuned scalars the effect is probably small, but a source-disjoint split would be stricter.
- *Dataset-specific tuning.* The kernel size and floor were tuned on 227×227 patches in which cracks fill a large part of the frame. Full-resolution inspection photographs, with joints, edges, shadows, rebar stains and different crack scales, will need re-tuning or a multi-scale formulation.
- *Label noise and class balance.* Some "clean" images show cracks (Section 5.5), and the balanced 50/50 classes do not reflect inspection practice, where cracks are rare; precision would be lower at a lower prevalence.
- *Single-scale scale calibration.* Millimetre conversion assumes a planar surface viewed head-on; oblique views or curved surfaces introduce perspective error.
- *Grid boundary.* The best kernel is at the edge of the searched range.

7. CONCLUSION

We presented a training-free pipeline that detects concrete cracks with black-hat morphology, rejects texture and stains with an absolute contrast floor and shape filters, and measures the length and width of each crack through its skeleton and distance transform. On 2,000 held-out METU images it achieves 95.5% accuracy, 96.5% recall and 94.5% precision, and its errors are interpretable: very wide cracks and heavily textured or pitted surfaces. Future work includes pixel-level evaluation on CrackForest, validation of measured widths against physical measurements, multi-scale black-hat kernels to handle wide cracks, a source-disjoint evaluation split, and testing on full-resolution inspection imagery.

REPRODUCIBILITY

From the project folder: `python tune.py` (grid search on the validation split), `python main.py --evaluate metu` (held-out evaluation and per-image predictions), `python experiments.py` (baselines and ablations). The random seed, split and all parameters are fixed in `config.py`.

REFERENCES

1. A. Mohan and S. Poobal, “Crack detection using image processing: A critical review and analysis,” *Alexandria Eng. J.*, vol. 57, no. 2, pp. 787–798, 2018.
2. L. Zhang, F. Yang, Y. D. Zhang, and Y. J. Zhu, “Road crack detection using deep convolutional neural network,” in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, 2016, pp. 3708–3712.
3. Ç. F. Özgenel and A. G. Sorguç, “Performance comparison of pretrained convolutional neural networks on crack detection in buildings,” in *Proc. 35th Int. Symp. Autom. Robot. Constr. (ISARC)*, Berlin, Germany, 2018.
4. Y. Shi, L. Cui, Z. Qi, F. Meng, and Z. Chen, “Automatic road crack detection using random structured forests,” *IEEE Trans. Intell. Transp. Syst.*, vol. 17, no. 12, pp. 3434–3445, 2016.
5. N. Otsu, “A threshold selection method from gray-level histograms,” *IEEE Trans. Syst., Man, Cybern.*, vol. 9, no. 1, pp. 62–66, 1979.
6. J. Canny, “A computational approach to edge detection,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. PAMI-8, no. 6, pp. 679–698, 1986.
7. K. Zuiderveld, “Contrast limited adaptive histogram equalization,” in *Graphics Gems IV*, P. S. Heckbert, Ed. San Diego, CA, USA: Academic Press, 1994, pp. 474–485.
8. J. Serra, *Image Analysis and Mathematical Morphology*. London, U.K.: Academic Press, 1982.
9. P. Soille, *Morphological Image Analysis: Principles and Applications*, 2nd ed. Berlin, Germany: Springer, 2003.
10. T. Y. Zhang and C. Y. Suen, “A fast parallel algorithm for thinning digital patterns,” *Commun. ACM*, vol. 27, no. 3, pp. 236–239, 1984.
11. G. Borgefors, “Distance transformations in digital images,” *Comput. Vis., Graph., Image Process.*, vol. 34, no. 3, pp. 344–371, 1986.
12. Ç. F. Özgenel, “Concrete crack images for classification,” Mendeley Data, v2, 2019. doi: 10.17632/5y9wdsg2zt.2.
13. E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.
14. Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
15. G. Bradski, “The OpenCV library,” *Dr. Dobb’s J. Softw. Tools*, 2000.
16. S. van der Walt *et al.*, “scikit-image: Image processing in Python,” *PeerJ*, vol. 2, p. e453, 2014.
17. C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.